

Stream Cipher & Randomness

Spring 2025

Stream cipher

- Remember that in OTP, key generation is hard.
 - Needs to have the same length as the message.
 - Must not be reused.
 - Takes too much space.
 - Sharing the key is also hard.
- People wanted to create an alternative.
 - A pseudorandom key stream.

Stream cipher

- In OTP we have perfect secrecy, but there are problems.
 - Key generation, distribution is hard.
 - We cannot reuse the same key.
- Now, instead of a **key**, we have a **seed** which helps us generate a new key.
 - That means we need a function which has an input (**seed**) and gives an output (**key stream**).
 - That is called **key stream generator**.
 - Creates a long pseudorandom sequence.
 - That generated key stream will act as **key** and XORed with the plaintext.

Stream cipher

- In short, instead of a long key, we select a **short key** and an **algorithm** which generates a *long key* from that *short key*
 - That short key is called a **seed**
- That seed is given to an algorithm.
 - The algorithm **always** creates the same long key for a given **seed**. (deterministic)
 - So, we are going to share the *seed* with the other party, and they can generate the key themselves.
 - **Instead of sending the whole key, we send the seed!**

Benefits

- Now we have a short key.
 - A 128-bit key is going to be sufficient instead of a message-length key.
- We can generate keys very fast.
- Key distribution is easier.
 - We are just sharing the seed now.
- However, it is not *true* random.
 - That is important.

Stream cipher

- Key setup
 - A short secret key is selected.
- Key stream generation
 - An algorithm expands the short key into a long keystream.
- Encryption
 - Each plaintext byte(bit) is XORed with the corresponding keystream byte(bit).

Stream cipher

- However, we don't have perfect secrecy anymore.
 - Because the key is not **actually** random.
 - It is **pseudorandom**
- Stream ciphers give up *perfect secrecy* for **good enough** secrecy and a much smaller secret key.

Stream ciphers

- RC4
 - Old and historical
 - Not used anymore.
 - Was in WEP, TLS, etc.
- Salsa20 / ChaCha20
 - Modern and secure

Nonce / IV

- If I encrypt two messages with the same secret key (seed), I get the same keystream.
 - **Very big problem**
- Use a public value that changes for **each encryption**.
 - So, even if we use the same secret key, we have a **different keystream**.
- Why?
 - Our aim is to use the **same long-term secret key** safely across multiple messages.
 - We use the same key but **different** nonce.

- Key K + nonce N1 $\rightarrow$ keystream S1
- Key K + nonce N2 $\rightarrow$ keystream S2

Randomness

- What makes a number, random?
 - Unpredictability
 - Uniform distribution
 - There should be **no pattern**.
 - All the numbers in the range are equally likely to occur.

True & Pseudo Random Number Generators

- Random numbers are divided into two categories
 - True Random Numbers (TRN)
 - Pseudo Random Numbers (PRN)
- **True random numbers** come from physical phenomena.
- **Pseudo random numbers** are created by mathematical calculations.

True Random Numbers

- Generally occur from physical phenomena.
 - e.g. Throwing a dice, lava lamp, time between keyboard strokes etc.
- You must not be able to fully recreate the same action. Therefore, it is **unpredictable**
- Software **cannot** produce true random numbers.
 - Because it uses an **algorithm**. It is **deterministic**.
 - They can only generate **pseudorandom numbers**.
- That means we need physical phenomena or hardware.

TRNG (True Random Number Generator)

- Lavarand (Wall of Entropy)
 - Hardware random number generator
 - Cloudflare has a wall of lava lamps.
- They take the pictures of the patterns made by the floating material in lava lamps.
- A seed is generated from these images.
 - [The Hardest Working Office Design In America Encrypts Your Fast Company](#)

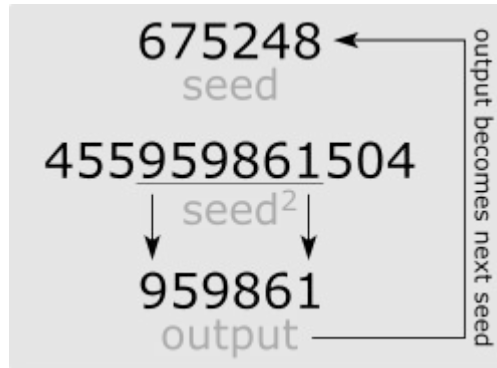


PRNG

- Created by mathematical functions.
- These functions can be *recreated*, they are *deterministic*. That means they are not truly random.
- You can create a *uniform distribution*
- We cannot use PRNG in cryptographic settings.
 - We **must** use CSPRNG.
- However, we still need PRNG.
 - Good for games, simulations, simple randomness.
 - Fast, because they don't need complex stuff.
 - Are predictable.

PRNG

- *Anyone who considers arithmetic methods of producing random digits is, of course, in a state of sin.*
- One of the first PRNG is created by [John Von Neumann](#).
 - [Middle-Square Method](#)
 - You choose a seed number.
 - Take the square.
 - Get the middle x digits.
- If you have the same seed (starting pt) you will have the same output everytime!



PRNG

- Different programming languages have different **random** implementations
 - Python -> *Mersenne Twister (MT19937)*
 - C -> *LCG (Linear Congruential Generator)*
 - $X_{n+1} = (a \cdot X_n + c) \bmod m$
 - There are also several more.
- These algorithms **must not be used** for cryptographic purposes.
 - Key, session, seed generation
 - OTP generation, password generation
- e.g. in Python you have two libraries: **random** and **secrets**
 - Secrets create CSPRNG (Cryptographically Secure PRNG)
 - Random create PRNG.
- Online, tutorials are using **random** library to build **password generators**
 - WRONG! They can be broken with enough consecutive data.

Seed

- Seed is the data that we start with.
 - In PRNG, if we have the same seed, we will always have the same output.
 - That is why in simulations, we use a certain seed.
 - Maybe you've seen it in data mining.
 - Why?
 - Because we want to replicate the same actions in different computers.
- How to create the seed?
 - In PRNG, it is not a problem.

Seed

- Let's open up two different shells and use Python random module with a seed: 42
 - Why 42?
- When we generate random numbers, it will have the same order!
 - `import random`
 - `random.seed(42)`
 - `random.randint(0,100)`
 - ..

CSPRNG

- For security purposes, we **must** use CSPRNG.
- Initialized with a high-entropy seed. Must be random. **(We cannot write our own seed here)**
 - Hardware sources, OS entropy pools, user input randomness, etc.
 - Linux: /dev/random
 - Windows: BCryptGenRandom
 - Entropy pools are reservoir of unpredictable data. OS constantly collects data (keyboard/mouse timing, network timing, disk activity etc.)
 - **Entropy:** unpredictability
 - That is why the lava wall is called: “*Wall of Entropy*”
- Different approaches:
 - Block cipher-based, Hash-based, HMAC based, LFSR based (but not traditional lfsr)

CSPRNG use cases

- Secure Tokens
 - password reset tokens
 - verification tokens
- Key generation
- Password generation

Statistical Independence

- **TRNG**

- Let's throw a die two times in a row.
- Does the second throw connected to the first one?
 - No. It means **independent**.

- **PRNG**

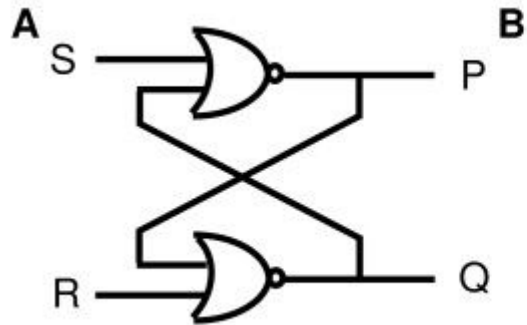
- Since we are using a mathematical function, it will have an order.
- There will be a pattern. Every new number is going to be connected to a previous number. So, they are **not** independent. They are **deterministic**.
- We can actually find them if we have some kind of information about the previous number.

Efficiency

- Efficiency is also can be a problem.
- Physical phenomena can be very slow.
 - Let's say we need a key of 256 bits. How many coin throws should we do?
 - If we want **true** randomness, we need to throw it at least 256 times.
- However, PRNG is very efficient compared to TRNG.
 - However, they are not **truly** random. They are **almost** random.

Latch

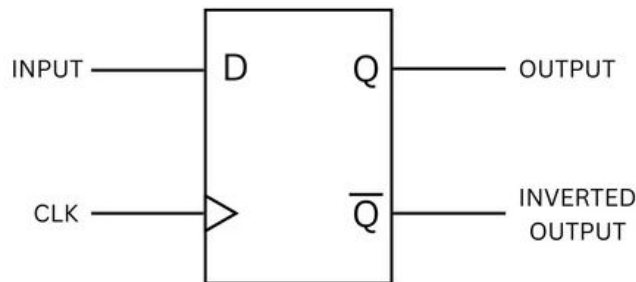
- Simplest form of memory.
- SR Latch (Set-Reset)
 - Two inputs: S and R
 - S -> Stores 1
 - R -> Stores 0
- So, if you don't do anything, output stays the same



Input		Output	
S	R	P	Q
0	0	Hold Output	
0	1	1	0
1	0	0	1
1	1	Invalid Input	

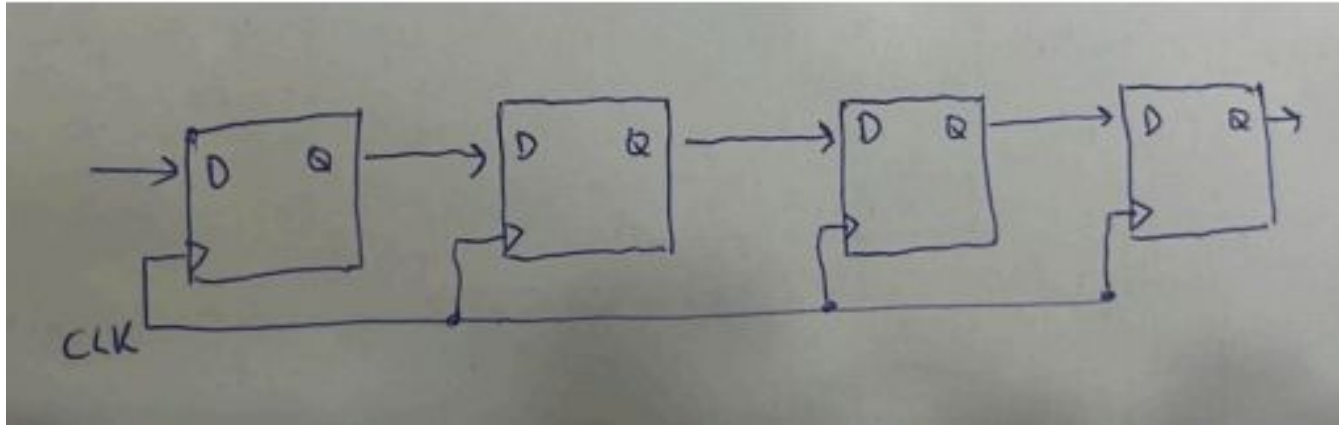
Flip-Flop

- In the latch, when you change an input the output changes.
 - Not exactly long term storage.
- D Flip-Flop solves this problem by updating only **on the clock signal**.
 - D (Data) : Value we want to store
 - CLK (Clock): Controls when the value is going to be updated.
- On a rising clock, Q becomes D.
 - Otherwise, Q stays the same.
- So, flip-flop stores the bit and changes only on the clock edge.
 - Latch would change automatically when the input changes.
- Useful for storing sequential data.



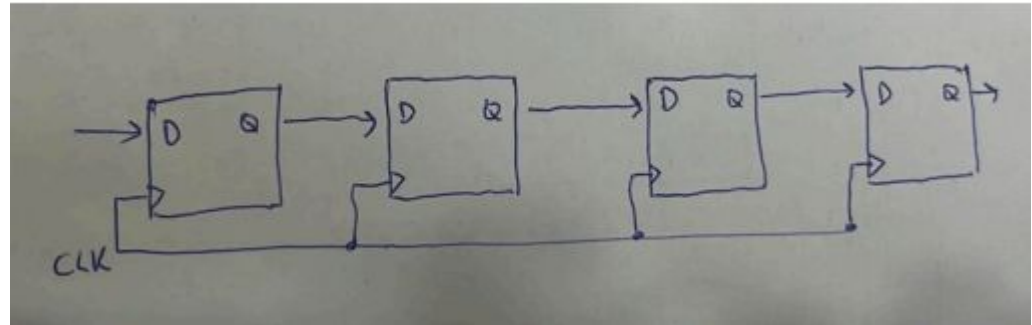
Flip-Flop

- If we have multiple FF's together, they will shift their values in every clock.



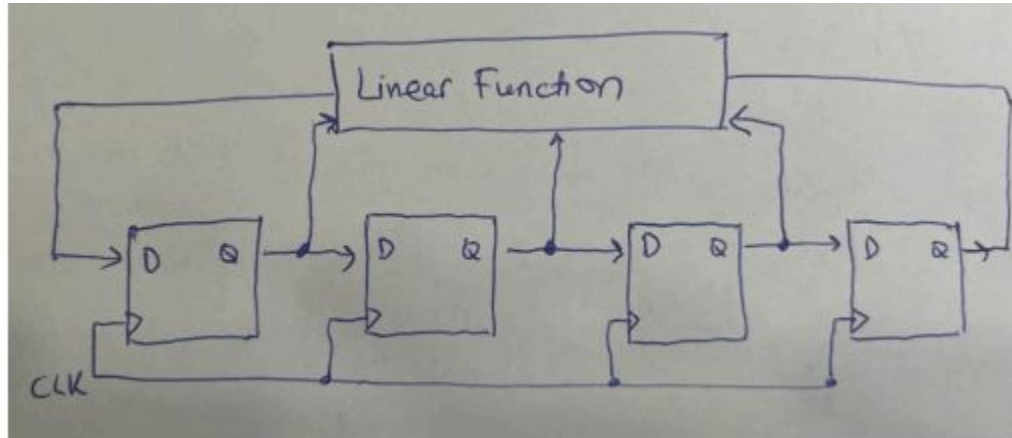
Shift Register

- A type of digital circuit using a cascade of FF (Flip Flops), where the output of a FF is connected to the input of the next.
- Used to store and move data.
 - Delaying signals
 - Temporary storage
 - To perform bitwise operations
 - etc.



LFSR: Linear Feedback Shift Register

- A shift register whose input bit is a linear function of its previous state.
- This is a 4-bit LFSR.
- The quality of LFSR depends on the Linear Function.



LFSR

- **LFSR's** are used as **PRNG's** for use in **stream ciphers**.
- Their output is used as the **keystream**.
- Easy to construct in hardware (electromechanical and electronic circuits)
- Used in **A5/1** and **A5/2**
 - GSM
- **Not good enough** for cryptography. The output can be **predicted** if enough bits of the sequence are known.
- In order to use them for cryptography, they are usually used in combination with other methods.

Example

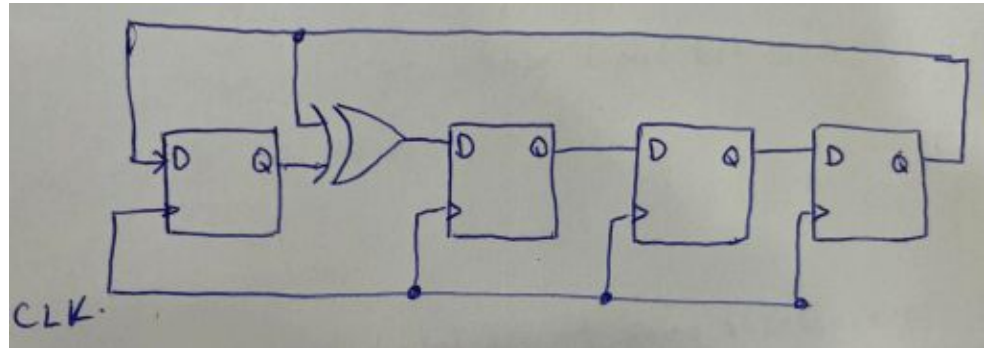
- Let's imagine a simple shift register.
 - The last bit is coming to first one and it is just shifting numbers.
 - We don't have any **linear function**.
 - Let's try 1001.
- Now, let's try the same number with a **linear function** now.
 - Use the image from below.
 - Rule : $x_3 = x_1 \text{ xor } x_0$
 - (seed) 1001 $\rightarrow$ x_3, x_2, x_1, x_0 (this is the order)
- When you do it, after 15 steps you will come back to your original, 1001.
 - It is inevitable to return to starting point.
 - Since we have 4 digits, the maximum amount you can do is $2^x - 1$.
 - Now you can choose 4-bits or 8-bits, combine them and have your random number.
- The complexity of the rule is not that important.
 - Change the rule and try again

Example

- Let's say we have a 4-bit number. The rightmost bit is going to be the *state*, which is the output. That will be a *bit* of our key.
- In a shift register, you are *shifting* digits.
 - However, this is a **LFSR**. That means, we are going to give the state to the first digit, by going through a **linear function** which consists of **XOR** operations.
 - That function is called a **rule** (or a **tap**).

Internal LFSR

- Previous examples were **external LFSR**.
 - Inputs of the FF's are not changed during the transfer.
 - Everything is shifted perfectly and we only played with the feedback.
- However, it is also possible that while shifting, we can apply some functions. That is an **internal LFSR**.
- Let's do an example with 1001.



Multiple LFSR

- LFSR's by themselves are not good enough for security.
 - We can combine two different LFSR's with different lengths and different rules.
 - Can use three or more LFSR and apply *majority voting*
 - Can use **irregular clocking**.
- In short, complexities can be added to have multiple LFSR.
- Irregular clocking
 - e.g. imagine there are 2 LFSR's. The clock of one LFSR is connected to the output (or maybe even a FF in the middle). So, if the output of the second LFSR is 1, clock is going to be activated and LFSR 1 will run for the next cycle.

Cryptanalysis (Known plaintext attack)

- One can find the keystream if the plaintext is known.
- From there, the key is analyzed so that the attacker can deduce the LFSR configuration.
- The complexity depends on how well the algorithm is
 - the length, the feedback function, etc.
- Several methods
 - Linear consistency method
 - Berlekamp-Massey algorithm

A5/1

- Uses three LFSR's. Here, a register is clocked if the clocking bit (orange) agrees with the clocking bit of one or both of the other two registers. At any clock cycle, the *majority vote* is checked. So, if the majority of oranges are 1, registers having 1 will advance. If the majority is 0, registers with 0 will advance.

